



LAMA

LEGACY APPLICATION
MODERNISATION AI STUDIO

Prerequisites & User Guide

Everything needed to go from a bare machine to a working migration — what to install, how to install it, and how to use the product once it is running.

INSTALL

Windows

Docker or local terminal, with the WSL 2, HOME and line-ending specifics.

INSTALL

macOS

Docker or local terminal, including Apple Silicon.

INSTALL

Linux

Docker Engine or local terminal, plus server hardening.

01

Discovery

02

Data Model

03

Architecture

04

CodeGen

05

Living

What is in this guide

GETTING INSTALLED

1	What LAMA is	The problem it solves, its features, the four project types
2	Prerequisites	Docker vs local, versions, what to have ready
3	Installation on Windows	Both paths, plus Windows-specific troubleshooting
4	Installation on macOS	Both paths, plus Apple Silicon notes
5	Installation on Linux	Both paths, plus running it as a team server
6	First boot and sign-in	Including the LLM provider setup, which is mandatory

USING THE PRODUCT

7	Using LAMA	The workspace, all five stages, the other project types, the admin pages
---	-------------------	--

KEEPING IT RUNNING

8	Day-to-day operations	Lifecycle, logs, backup, upgrade
9	Troubleshooting	General, plus a table per platform
10	Reference	Settings, ports, glossary, verification notes

Each operating-system section covers **both** installation paths — Docker and local terminal — and ends with troubleshooting specific to that platform. Read only your own.

What LAMA is

1.1 The problem it exists to solve

Most legacy systems cannot be described by anyone still working on them. The people who wrote them have gone, the documentation was never written or is years out of date, and the only authoritative statement of what the system does is the source code itself — hundreds of thousands of lines of PHP, JSP, Struts or classic .NET, with business rules buried in controllers, validation duplicated across screens, and a database whose constraints live in application code rather than in the schema.

That makes modernisation genuinely risky. A rewrite has to reproduce behaviour nobody has written down. Teams typically spend months on manual discovery, produce a requirements document that is already incomplete, and then discover the gaps only after the new system is in production.

LAMA attacks the discovery problem first, and treats code generation as the last step rather than the first. It reads your legacy source, extracts what the system actually does into a reviewable knowledge base, turns that into a formal requirements specification you can correct, and only then designs the data model, the services and the code. At every step the evidence stays attached to the claim, so you can see *why* LAMA believes a requirement exists.

1.2 How it helps you, concretely

THE PROBLEM	WHAT LAMA DOES ABOUT IT
"Nobody knows what this system does any more."	Parses the source into a Knowledge Base — files, entities, routes, tables, columns, roles, and the relationships between them — then generates a 12-section IEEE-830 SRS grounded in that evidence.
"The documentation we do have is fiction."	Every generated claim is traceable to the source that produced it. Where the code does not support a claim, LAMA marks the section EVIDENCE GAP rather than inventing something plausible.
"How do we know the spec is good enough to build on?"	A confidence engine scores every section for how well the evidence supports it, flags contradictions, and tells you which sections to review — before you commit to them.
"The rewrite drifted from the requirements."	Stages are frozen and downstream work is generated from the frozen record, not from a moving target. Drift detection later compares the running system back against the frozen spec.
"We can't tell what the new code is missing."	Gap analysis produces a coverage matrix of requirements against implementation, with a severity-ranked list of what is absent.
"Migrating the database by hand will take months."	Generates target OLTP DDL , an OLAP star schema , a bus matrix, and three migration scripts: legacy→OLTP, OLTP→OLAP, and a migration test.
"Generated code never compiles."	The compile-and-repair loop actually builds the output and escalates through four repair strategies. A missing toolchain fails the job rather than being skipped, so nothing can report success without having compiled.
"We just need this one stack moved to another stack."	Two dedicated transformation tools, one of which needs no requirements work at all.

1.3 Capabilities shared across the whole product

These apply whichever project type you choose.

CAPABILITY	WHAT IT MEANS FOR YOU
Evidence-grounded generation	Prompts are built from the knowledge base, not from the model's general knowledge of "how PHP apps usually work".
Bring your own model	OpenRouter, Anthropic, OpenAI, Azure, Gemini, Groq, or a local Ollama. Nothing is hard-coded, and a fully local Ollama setup runs offline.
Per-agent model routing	Each internal agent declares a complexity tier, and you map tiers to models. Cheap models for chat, your best coding model for code generation — without configuring anything per-agent.
Token-spend visibility	Usage is reported by provider, model, stage and agent over a date window.
Human-in-the-loop everywhere	Every artifact can be read, hand-edited, regenerated section by section, downloaded, and versioned. LAMA is a drafting tool with gates, not a black box.
Typed-confirmation gates	Freezing and resetting require typing <code>FREEZE</code> or <code>RESET</code> . Destructive and load-bearing actions are deliberately hard to do by accident.
Live logs	A MiniConsole streams backend activity on every long-running page, with a Copy button.
Multi-tenancy and audit	Tenants, users and roles, with every state-changing action recorded in an audit log.
Export anywhere	ZIP download, export to a folder on disk, or push straight to GitHub.

1.4 The four project types

When you click **New Project**, LAMA asks you to **Choose Project Type**. The choice decides which stages appear in the sidebar and which engine runs. All four ship in the product.

	TYPE	TAGLINE	STAGES	NEEDS A KNOWLEDGE BASE?
1	Legacy Modernization	PHP/JSP/.NET → FastAPI/Node/Spring	5	Yes
2	Gap Analyzer	SRS ↔ Source code coverage	3	Yes
3	Technology Transformer	Cross-stack transformation	3	Yes
4	Direct Transform	Folder → migrated tree, no KB	3	No

1.4.1 Legacy Modernization

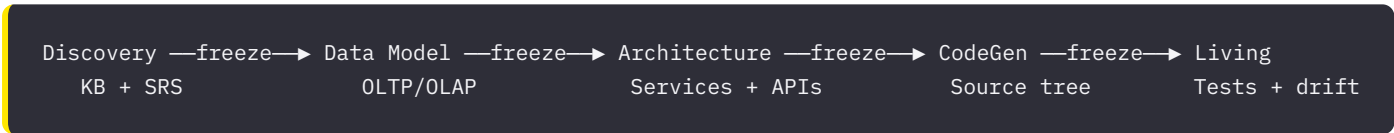
PHP/JSP/.NET → FastAPI/Node/Spring

Full 5-stage pipeline: Discovery → DataModel → Architecture → CodeGen → Living.

The complete modernisation workflow, and the reason the rest of the product exists. Use it when you are rebuilding a legacy application as a new cloud-native system and need the requirements, the data model and the architecture to be defensible — not just the code.

It is a **strictly sequential five-stage pipeline**. You do not jump from legacy source straight to generated code. Each stage must be reviewed and **frozen** before the next unlocks, and freezing writes a "Stage Context" that becomes the single source of truth downstream. That is what stops a code generator from inventing requirements out of half-read evidence.

#	STAGE	WHAT YOU PRODUCE	FREEZING IT UNLOCKS
1	Discovery	Knowledge Base, YAML/TOON context, business ontology, a 12-section IEEE-830 SRS	Data Model
2	Data Model	PostgreSQL OLTP DDL, OLAP star schema, Bus Matrix, 3 migration scripts	Architecture
3	Architecture	Service Map, HLD, LLD, OpenAPI 3.1 contracts, sequence diagrams	Code Generation
4	Code Generation	Per-service source tree, Dockerfiles, ZIP, disk export, GitHub push	Living System
5	Living System	Selenium tests, JMeter plans, SRS-drift report, accuracy report	—



Features

- Four ways to import evidence: upload files or a ZIP, scan a folder on the server, clone a Git repository, or connect a live database and application URL.
- Language-agnostic extraction — PHP, JSP, .NET, Python, JavaScript and more.
- A business ontology and property graph built alongside the raw knowledge base.
- 12 IEEE-830 SRS sections, generated in parallel, individually regenerable and editable, exportable to PDF.
- Confidence scoring per section, with contradiction detection and a hard stop.
- Chat grounded in the knowledge base, which can also apply edits straight into the SRS.
- Both operational and analytical schemas, plus the scripts to move the data.
- Service decomposition you approve, merge and split before any code is generated.
- Gap recovery and an auto-validate loop that scores generated files and repairs the worst.
- Selenium and JMeter generation, SRS diffing and drift detection after go-live.

Skipping. Data Model, Architecture and Living can be skipped from the sidebar menu when they do not apply — that records a skip and unlocks the next stage. Discovery and Code Generation cannot be skipped.

1.4.2 Gap Analyzer

SRS ↔ Source code coverage

KB-driven verification of requirements against code. Produces coverage matrix + gap report.

Not a migration tool — a verification tool. Use it when the code already exists and the question is whether it does what the specification says. That covers accepting a vendor delivery, auditing your own build against its requirements, and re-checking coverage before a release.

STAGE	WHAT HAPPENS
1. Input Files	Upload the source code and the SRS/FRS document to check it against.
2. Knowledge Base	LAMA extracts UI → API → DB traceability and a structured requirement list.
3. Gap Report	A coverage matrix plus a severity-ranked list of requirements the code does not implement.

Features

- Requirements parsed out of your existing SRS/FRS — no need to re-author it in LAMA.
- End-to-end traceability: which screen calls which endpoint, which touches which table.
- Severity ranking, so you can tell a missing audit trail from a missing tooltip.
- Its own **freeze / unfreeze**, independent of the five-stage pipeline.
- Export the report for circulation.

It requires no frozen upstream stage. You can run it on day one.

1.4.3 Technology Transformer

Cross-stack transformation

KB-anchored per-file transformation between technology stacks with dependency preservation.

Use it when the **application design is fine** and only the technology has to change — a stack upgrade or a platform move, where re-deriving requirements would be wasted effort, but you still want the transformation anchored in an understanding of the code rather than done file-by-file in isolation.

STAGE	WHAT HAPPENS
1. Input Files	Upload the source code and choose the target stack.
2. Knowledge Base	Entities and a dependency graph are extracted.
3. Transformed	The per-file transformed source tree, with downloads.

Features

- **Dependency preservation** — the dependency graph is built first, so transformation respects real relationships instead of treating each file as standalone.
- A **multi-agent pipeline**: planner, coder, verifier, reviewer and tester, with a visible agent timeline and a plan you confirm before it runs.
- A **compile-and-repair loop** that escalates through four strategies — coder, then a DevOps expert, then the same expert with the raw build log, then a full regenerator — capped at five iterations.
- The **target manifest is derived from the migrated code's imports**, the way an IDE resolves an unresolved import, rather than being guessed.

- Per-file **regenerate**, pause, resume and stop on a running job.
- Live log streaming, a traceability view, and an agent chat.
- ZIP download and direct GitHub push.
- Target-stack idioms are table-driven, so supporting another language is a configuration entry rather than new code.

NOTE

The build state is a report, not a permission. `compile_green` and `production_ready` tell you where the build stands. They do **not** block download or push — you can always collect your own code, green or not.

1.4.4 Direct Transform

Folder → migrated tree, no KB

Point it at a folder on the server: deterministic plugins first (Helidon → Spring Boot, Oracle → PostgreSQL), AI pass for every other stack pair.

The **fastest** path, and the only one that builds no knowledge base at all. Use it when you know exactly what you have and exactly what you want, and you want the conversion rather than the analysis. It is folder-path driven: you point it at a directory on the machine running LAMA and it writes a migrated tree to another directory.

STAGE	WHAT HAPPENS
1. Configure	Pick the source folder, the source and target stacks, and the destination.
2. Transform	Run the job and watch the engine and agent event stream.
3. Output	Reports and the per-file transform record.

Source and target are chosen independently from one catalogue:

FAMILY	STACKS
Backend	Helidon MicroProfile, Helidon SE, Spring Boot 4.1 (Java 25 LTS), Spring Boot 3.x (Java 21), .NET 10 LTS, JSP / Jakarta Pages 4.0, Apache Struts, EJB 3.x
Frontend	React 19, Angular 22, jQuery
Database	Oracle (SQL / PL-SQL), PostgreSQL 18, PostgreSQL 15

Features

- **Deterministic plugins first.** Certain pairs — Helidon MP → Spring Boot 3, Oracle → PostgreSQL — are converted by hand-written transformers that produce the same output every time. Every other pair falls back to an AI pass. The UI tells you which one your chosen pair will get **before** you start.
- **The catalogue drives the engine, not just the prompt.** Each stack declares the file types it owns, the markers that must not survive migration, and its build command — so the engine sweeps the right files, detects half-migrated ones, and builds with the right toolchain.
- **Per-pair migration briefs.** The instructions sent to the model are composed for your specific source→target pair.
- **A no-residue gate** that detects files still carrying the source stack's constructs.

- Pause, resume and **rollback** on a running job.
- A boot smoke test that starts the migrated application and probes its health endpoint.
- CI/CD scaffolding and a downloadable artifact.

NOTE

Because it has no knowledge base, it has no requirements view. Direct Transform converts what is in the folder. If you need to know whether the result still satisfies the specification, that is the Gap Analyzer's job.

1.5 Which type should I choose?

IF YOUR SITUATION IS...	CHOOSE
Rebuilding a legacy system, and the requirements need to be recovered and defensible	Legacy Modernization
The code exists; you need to know what it does not implement	Gap Analyzer
The design is fine, the technology must change, and you want it anchored in the code's real structure	Technology Transformer
You know the source stack and the target stack, and just want the conversion	Direct Transform

They are not exclusive. A common sequence is **Direct Transform** or **Technology Transformer** to move the stack, then **Gap Analyzer** to prove the result still satisfies the specification.

1.6 What LAMA needs from the outside world

LAMA does not ship a model. Every generation call is routed to an **LLM provider you configure** — OpenRouter, Anthropic, OpenAI, Azure, Gemini, Groq, or a local Ollama. There are no hard-coded vendor defaults anywhere in the product: an install with no provider configured will raise an error rather than quietly guess a model. **Configuring a provider is a mandatory installation step, not an optional one.**

Prerequisites (all platforms)

2.1 Choose your installation path

	PATH A — DOCKER	PATH B — LOCAL TERMINAL
Best for	Evaluating, demos, shared team servers, anyone who is not editing LAMA itself	Developers changing LAMA, or machines where Docker is not allowed
You must install	Docker Desktop (or Docker Engine) — that's all	Python, Node + yarn, MongoDB
MongoDB	Bundled inside the container	You install and run it
Where it runs	Everything on <code>http://localhost:8382</code>	API on <code>:8000</code> , UI on <code>:3000</code>
After a code change	Restart the container	Hot reload, automatic
Disk	~6 GB (image + volumes)	~4 GB (venv + node_modules)

If you are unsure, choose Path A. Most of this guide's platform differences disappear when everything runs in one container.

2.2 Hardware and network

REQUIREMENT	MINIMUM	COMFORTABLE
RAM available to LAMA	4 GB	8 GB
Free disk	10 GB	25 GB
CPU	4 cores	8 cores
Free TCP port	<code>8382</code> (Docker) — or <code>8000</code> + <code>3000</code> (local)	
Network	Outbound HTTPS to your LLM provider	Also to Docker Hub, PyPI, npm on first install

If you run everything locally with Ollama, LAMA works **fully offline** after the first install. The vector store has an embedded on-disk mode that needs no server, and the confidence engine's models are cached to disk.

2.3 Software versions — do not substitute

These versions are not arbitrary. Where a version is pinned, an older one does not fail cleanly; it produces broken output that reads like a bad migration.

SOFTWARE	VERSION	NEEDED FOR	NOTES
Docker	Any current release with Compose v2	Path A	<code>docker compose version</code> must work — the space-separated form, not <code>docker-compose</code>
Python	3.11+ (3.14 works locally)	Path B backend	The container runs 3.11
MongoDB	7 or 8	Path B — the system of record	Bundled in the Docker image
Node.js	20+ (image builds on 24 LTS)	Path B frontend	
yarn	1.22 via <code>corepack enable</code>	Path B frontend	Never run <code>npm install</code> in <code>frontend/</code> — the lockfile is yarn's
An LLM provider	—	Everything	Ollama locally, or an OpenRouter/Anthropic/OpenAI key

Optional build toolchains

Only the compile-and-verify agents in Code Generation, Transformer and Direct Transform use these. The pipeline itself runs without them.

TOOLCHAIN	NEEDED WHEN YOUR TARGET IS	VERSION THAT MATTERS
JDK	Spring Boot 4.1	Temurin/JDK 25 — JDK 17 or 21 cannot compile what the <code>spring-boot-4</code> target emits
.NET SDK	.NET 10	SDK 10 — SDK 8 cannot build <code>net10.0</code>
Maven / Gradle	any JVM target	current
Node 20+	React 19 / Angular 22 targets	
Go	Go targets	

WATCH OUT

A missing toolchain is a hard failure, by design. The compile agent returns `failed` when a binary is absent rather than skipping the check — so a build can never report "compilation ready" without having really compiled. The Docker image ships **Temurin 25** and **.NET SDK 10** already.

Optional services

SERVICE	GIVES YOU	WITHOUT IT
Qdrant (vector DB)	Semantic KB search, RAG-grounded chat, SRS evidence retrieval	LAMA degrades silently to plain-text context. Everything still works, retrieval is just less precise. Embedded on-disk mode needs no server.

SERVICE	GIVES YOU	WITHOUT IT
Factory / droid CLI	Optional alternate execution path for agents	Not needed; leave it off

2.4 What you need before you start

Have these ready:

- 1. **An LLM API key** — e.g. an OpenRouter key beginning `sk-or-...` — *or* Ollama installed locally.
- 2. **A random string, 32+ characters**, for `LAMA_JWT_SECRET`. This signs login tokens. Anything random will do; do not ship the placeholder.
- 3. **A copy of the legacy application** you intend to migrate, on disk.
- 4. If your organisation intercepts TLS (Zscaler, Netskope, and similar): **your corporate root CA in PEM form**.

Installation on Windows

Windows needs three things the other platforms do not: the WSL 2 backend, an explicit `HOME` variable, and LF line endings. Each is covered below.

All commands in this section are **PowerShell**. Open it from Start → "Windows PowerShell".

3.1 Install the prerequisites

Path A (Docker) — install one thing

1. Download **Docker Desktop** from <https://docker.com/products/docker-desktop>.
2. During install, keep "Use WSL 2 instead of Hyper-V" ticked.
3. After install, open Docker Desktop → Settings → General and confirm "Use the WSL 2 based engine" is on.
4. Verify:

```
docker version --format '{{.Server.Version}}'
docker compose version
```

Both must print a version. If `docker compose version` fails but `docker-compose --version` works, you have the old v1 binary — update Docker Desktop.

Path B (local terminal) — install the stack

WATCH OUT

Strong recommendation: on Windows, run Path B inside **WSL 2 (Ubuntu)** and follow [Part 5 — Linux](#) instead. LAMA's helper scripts (`setup.sh`, `doctor.sh`, `run-backend.sh`, `run-frontend.sh`) are bash scripts and do not run in PowerShell. Everything below is the manual equivalent if you must stay in native Windows.

Install, in order:

```
winget install --id Python.Python.3.12 -e
winget install --id OpenJS.NodeJS.LTS -e
winget install --id MongoDB.Server -e
winget install --id Git.Git -e
```

Then enable yarn and confirm each one:

```
corepack enable

python --version      # 3.11 or newer
node --version        # 20 or newer
yarn --version        # 1.22.x
git --version
```

MongoDB installed via winget registers a Windows service. Start it and confirm it is listening:

```
Start-Service MongoDB
Test-NetConnection 127.0.0.1 -Port 27017      # TcpTestSucceeded : True
```

Optional toolchains, only if you will generate code for those targets:

```
winget install --id EclipseAdoptium.Temurin.25.JDK -e
winget install --id Microsoft.DotNet.SDK.10 -e
winget install --id Apache.Maven -e
```

3.2 Get the code, with LF line endings

This matters and it is the most common Windows failure. Docker Compose bind-mounts a shell script into the container and runs it with `/bin/sh`. With Git's Windows default (`core.autocrlf=true`) that file arrives as CRLF, the shebang becomes `#!/bin/sh\r`, and the container dies with:

```
exec /entrypoint-with-wheels.sh: no such file or directory
```

— which names the script, not the carriage return, so it reads like a missing file. The repository ships a `.gitattributes` that forces LF, so a **fresh clone is correct**. Clone fresh:

```
git clone <your-repo-url> lama
cd lama
```

If you cloned *before* `.gitattributes` existed, renormalise instead:

```
git rm --cached -r .
git reset --hard
```

3.3 Create the paths Compose expects

Compose bind-mounts four host paths that are not in a fresh clone. A bind-mount whose source is missing is created by Docker as a **directory**, even when a *file* was expected — so a missing certificate silently becomes a folder and Nginx serves an empty site. Create them first:

```
New-Item -ItemType Directory -Force -Path certs, hf_cache, wheels, frontend\build | Out-Null
New-Item -ItemType File -Force -Path certs\corp-ca.pem | Out-Null
```

PATH	WHAT IT IS	IF YOU DO NOT USE THE FEATURE
<code>frontend\build</code>	The compiled UI bundle Nginx serves	Must be built — see 3.4
<code>certs\corp-ca.pem</code>	Your corporate root CA	An empty file is fine

PATH	WHAT IT IS	IF YOU DO NOT USE THE FEATURE
<code>hf_cache</code>	Offline models for confidence scoring	An empty folder is fine
<code>wheels</code>	Offline LangGraph wheels	An empty folder is fine

3.4 Build the UI bundle — inside Docker, no Node needed

`frontend/build/` is gitignored, so a fresh clone does not have it. Compose mounts your local copy **over** the one baked into the image, so if it is empty you get a **blank page at :8382 with a perfectly healthy container** — which looks like a broken app rather than a missing bundle.

Build it in a throwaway container (no Node on your machine required):

```
docker run --rm -v "${PWD}/frontend:/app" -w /app -e CI=true -e DISABLE_ESLINT_PLUGIN=true node:24-bookworm-slim sh -c 'corepack enable && yarn install --frozen-lockfile && yarn build'
```

Keep the inner command in **single** quotes — PowerShell then passes it through literally, which is what `sh -c` needs.

First run takes a few minutes (yarn install); about 30 seconds thereafter, since `node_modules` persists in the mounted folder.

(Alternatively, on Path B where you already have Node: `cd frontend; yarn install; yarn build; cd ..`)

3.5 Set `HOME` — Compose needs it, Windows does not have it

`docker-compose.yml` mounts `${HOME}/.local/bin-linux/droid` and `${HOME}/.factory` for the optional Factory CLI. Windows sets `USERPROFILE`, not `HOME`, so Compose warns *"The HOME variable is not set. Defaulting to a blank string"* and resolves those mounts to `/.local/bin-linux/droid` — the root of your drive.

Add to `.env` in the repository root:

```
HOME=C:/Users/<your-username>
```

Forward slashes, no trailing slash, no quotes.

If you do not use Factory at all, pin the three paths somewhere harmless instead:

```
LAMA_DROID_BIN_HOST=./certs/corp-ca.pem
LAMA_DROID_BIN_HOST_DIR=./certs
LAMA_FACTORY_CONFIG_HOST=./certs
```

3.6 Configure `.env`

Open `.env` in the repository root and set, at minimum:

```
OPENROUTER_API_KEY=sk-or-...
LAMA_JWT_SECRET=<your own random 32+ character string>
LAMA_FACTORY_MODE=
LAMA_DEFAULT_MODEL=
```

SETTING	WHY
LAMA_JWT_SECRET	Signs login tokens. Setting it here is not enough for the container — see 6.4 Securing the login .
LAMA_FACTORY_MODE=	Blank. The committed default is <code>cli</code> , which requires the optional <code>droid</code> binary inside the container; without it, every LLM call fails.
LAMA_DEFAULT_MODEL=	Leave empty . Any value here overrides per-agent tier routing for <i>every</i> call, so the Console's routing stops working.

Using Ollama on your Windows host instead of a cloud key? Leave `OPENROUTER_API_KEY` blank and add the provider from the UI after boot; Compose already maps `host.docker.internal`.

3.7 Start it

The committed `.env` pins `LAMA_IMAGE=lama:local` with `LAMA_PULL_POLICY=never`, which means Compose expects an image **you built**. Pick one option:

Option A — pull the published image (fastest)

```
$env:LAMA_IMAGE="mishramesh/lama:latest"
$env:LAMA_PULL_POLICY="always"
docker compose up -d
```

Option B — build locally

```
docker build -f Dockerfile.local -t lama:local .    # thin: adds git + dulwich
docker build -t lama:local .                        # full build from source
docker compose up -d
```

Windows hosts are `amd64`, so no `--platform` flag is needed.

Then watch it come up:

```
docker compose logs -f lama      # Ctrl-C stops following, not the container
curl.exe http://127.0.0.1:8382/health
Start-Process http://localhost:8382
```

NOTE

Use `curl.exe`, not `curl` — in PowerShell, `curl` is an alias for `Invoke-WebRequest`, which takes completely different flags.

First boot takes roughly 40 seconds while it seeds MongoDB, the prompt library and the agent configuration.

3.8 Path B — running natively on Windows

If you are not using Docker, run the two processes in two PowerShell windows. There is no `setup.sh` equivalent, so do it by hand once:

```
# one-time setup
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt

corepack enable
cd frontend; yarn install; cd ..
```

Create `backend\.env` (this is a **different file** from the repository-root `.env`, which belongs to Compose):

```
MONGO_URL=mongodb://127.0.0.1:27017
DB_NAME=lama
LAMA_JWT_SECRET=<your own random 48+ character string>

LAMA_OLLAMA_BASE_URL=http://localhost:11434/v1
LAMA_DEFAULT_MODEL=
OPENROUTER_API_KEY=sk-or-...

LAMA_CONFIDENCE_ENGINE=fabric

QDRANT_PATH=../qdrant_local
QDRANT_URL=
LAMA_EMBED_BACKEND=ollama
LAMA_OLLAMA_EMBED_MODEL=nomic-embed-text
```

Then, in two windows:

```
# window 1 — API
cd backend
..\venv\Scripts\uvicorn.exe server:app --reload --port 8000
```

```
# window 2 — UI
cd frontend
yarn start
```

Open <http://localhost:3000>.

3.9 Windows-specific troubleshooting

SYMPTOM	CAUSE AND FIX
<code>exec /entrypoint-with-wheels.sh:</code> no such file or directory	CRLF line endings. See 3.2 — reclone, or <code>git rm --cached -r . ; git reset --hard</code> .
The "HOME" variable is not set	See 3.5 — set <code>HOME</code> in <code>.env</code> .

SYMPTOM	CAUSE AND FIX
Blank page at :8382, container healthy	<code>frontend\build</code> is empty. Build the bundle (3.4), then <code>docker compose restart lama</code> .
<code>curl : A parameter cannot be found that matches parameter name 'fsS'</code>	You used PowerShell's <code>curl</code> alias. Use <code>curl.exe</code> .
<code>pull access denied for lama</code>	<code>.env</code> pins <code>LAMA_IMAGE=lama:local</code> and you have not built it. Use Option A in 3.7.
<code>Start-Service MongoDB</code> → service not found	MongoDB installed without the service option. Reinstall, or run <code>mongod.exe --dbpath C:\data\db</code> in its own window.
Docker Desktop won't start / "WSL 2 is not installed"	Run <code>wsl --install</code> in an elevated PowerShell, reboot, then start Docker Desktop.
Very slow file access under <code>frontend/</code>	Bind-mounted Windows folders are slow through WSL. Clone the repository inside the WSL filesystem (e.g. <code>\\wsl\$\\Ubuntu\\home\\you\\lama</code>) and work from there.

Installation on macOS

The macOS-specific point is **Apple Silicon**: the LAMA image is `linux/amd64`, and while it runs perfectly well under emulation on M-series Macs, building it yourself requires an explicit platform flag.

All commands in this section are for **Terminal** (zsh).

4.1 Install the prerequisites

Path A (Docker) — install one thing

Download **Docker Desktop** from <https://docker.com/products/docker-desktop>, choosing the Apple Silicon or Intel build to match your Mac. Then:

```
docker version --format '{{.Server.Version}}'
docker compose version
```

That is the entire prerequisite list for Path A. No Python, no Node, no MongoDB.

Path B (local terminal) — install the stack

Using Homebrew:

```
brew install python@3.12 node
brew tap mongodb/brew && brew install mongodb-community@8.0
corepack enable                # provides yarn 1.22.22
```

Verify:

```
python3 --version    # 3.11 or newer
node --version       # 20 or newer
yarn --version       # 1.22.x
```

Optional toolchains, only if you will generate code for those targets:

```
brew install --cask temurin@25    # JDK 25 — required by the spring-boot-4 target
brew install --cask dotnet-sdk    # verify it is SDK 10
brew install maven gradle go
```

4.2 Get the code

```
git clone <your-repo-url> lama
cd lama
```

4.3 Create the paths Compose expects (Path A)

Compose bind-mounts four host paths that a fresh clone does not contain. A bind-mount whose source is missing is created by Docker as a **directory**, even when a *file* was expected — so a missing certificate silently becomes a folder. Create them first:

```
mkdir -p certs hf_cache wheels frontend/build
touch certs/corp-ca.pem
```

PATH	WHAT IT IS	IF YOU DO NOT USE THE FEATURE
frontend/build	The compiled UI bundle Nginx serves	Must be built — see 4.4
certs/corp-ca.pem	Your corporate root CA	An empty file is fine
hf_cache	Offline models for confidence scoring	An empty folder is fine
wheels	Offline LangGraph wheels	An empty folder is fine

4.4 Build the UI bundle

frontend/build/ is gitignored, and Compose mounts your local copy **over** the one baked into the image. If it is empty you get a **blank page at :8382 with a healthy container** — which looks like a broken app rather than a missing bundle.

Path A (no Node on your Mac): build it in a throwaway container.

```
docker run --rm \
  -v "$(pwd)/frontend:/app" -w /app \
  -e CI=true -e DISABLE_ESLINT_PLUGIN=true \
  node:24-bookworm-slim \
  sh -c 'corepack enable && yarn install --frozen-lockfile && yarn build'
```

Path B (Node already installed):

```
cd frontend && yarn install && yarn build && cd ..
```

Either way, frontend/build/ is written. First run takes a few minutes; about 30 seconds thereafter, since node_modules persists.

4.5 Configure .env (Path A)

Open .env in the repository root and set, at minimum:

```
OPENROUTER_API_KEY=sk-or-...
LAMA_JWT_SECRET=<your own random 32+ character string>
LAMA_FACTORY_MODE=
LAMA_DEFAULT_MODEL=
```

SETTING	WHY
LAMA_JWT_SECRET	Signs login tokens. Setting it here is not enough for the container — see 6.4 Securing the login .
LAMA_FACTORY_MODE=	Blank. The committed default is <code>cli</code> , which requires the optional <code>droid</code> binary inside the container; without it, every LLM call fails.
LAMA_DEFAULT_MODEL=	Leave empty . Any value here overrides per-agent tier routing for <i>every</i> call, so the Console's routing stops working.

To reach an Ollama running on your Mac from inside the container:

```
LAMA_OLLAMA_BASE_URL=http://host.docker.internal:11434/v1
```

WATCH OUT

Back up `backend/.env` before your first Docker run. The container's entrypoint writes a runtime `.env` to `/app/backend/.env`, and Compose bind-mounts `./backend` there — so the write lands on your **host** file and replaces any local-dev settings you had: `bash cp backend/.env backend/.env.mine # restore with: cp backend/.env.mine backend/.env` This only matters if you have also used Path B on the same clone.

4.6 Start it (Path A)

The committed `.env` pins `LAMA_IMAGE=lama:local` with `LAMA_PULL_POLICY=never`, so Compose expects an image **you built**. Pick one:

Option A — pull the published image (fastest)

```
LAMA_IMAGE=mishramesh/lama:latest LAMA_PULL_POLICY=always docker compose up -d
```

Option B — build the thin local image (adds `git` + `dulwich` on the published base)

```
docker build --platform linux/amd64 -f Dockerfile.local -t lama:local .
docker compose up -d
```

Option C — build the full image from source

```
docker build --platform linux/amd64 -t lama:local .
docker compose up -d
```

WATCH OUT

`--platform linux/amd64` is required on Apple Silicon. A native arm64 build dies at the MongoDB layer with `E: Unable to locate package mongodb-org-server`, because MongoDB publishes no arm64 Debian packages on the `mongodb-org/7.0` channel. The error names the package, not the architecture, so it reads as a broken Dockerfile. Every other layer builds fine on arm64.

Then:

```
docker compose logs -f lama           # Ctrl-C stops following, not the container
curl http://127.0.0.1:8382/health     # → {"ok":true}
open http://localhost:8382
```

First boot takes roughly 40 seconds while it seeds MongoDB, the prompt library and the agent configuration. The healthcheck allows for that.

4.7 Path B — running natively on macOS

One-shot setup

```
./scripts/setup.sh
```

That creates `.venv` at the repository root, installs backend and frontend dependencies, and writes a starter `backend/.env` with a freshly generated JWT secret. It is safe to re-run: it never overwrites an existing `backend/.env`.

► What it does, if you prefer to run the steps yourself

WATCH OUT

Why `requirements-dev-macos.txt`? `requirements.txt` targets the container — Linux on Python 3.11 — and carries the HuggingFace/torch stack (~3 GB) that local development does not need. The `-dev-macos` file installs the same application at the same versions wherever a newer Python allows, and documents each relaxed pin inline. `requirements.txt` remains the source of truth for the Docker image.

Start MongoDB

```
brew services start mongodb-community@8.0
nc -z 127.0.0.1 27017 && echo "Mongo up"
```

WATCH OUT

The Homebrew formula is **versioned**. `brew services start mongodb-community` fails with "Formula not installed", and `brew services stop mongodb-community` reports "not started" even while it is running. Always use the `@8.0` suffix.

Check the environment

```
./scripts/doctor.sh
```

Read-only. It reports what is present, what is missing, and whether each gap actually blocks you — including the **versions** of the JDK and .NET SDK, not just their presence.

Run it — two terminals

```
./scripts/run-backend.sh           # API → http://127.0.0.1:8000/api/health
```

```
./scripts/run-frontend.sh    # UI → http://localhost:3000
```

URL	WHAT
http://localhost:3000	the application
http://127.0.0.1:8000/api/health	<code>{"ok":true}</code>
http://127.0.0.1:8000/api/health/providers	which LLM provider is live
http://127.0.0.1:8000/docs	interactive API reference

NOTE

Be consistent about `localhost` vs `127.0.0.1` between the UI and `REACT_APP_BACKEND_URL`. Browsers treat them as **different origins**, and mixing them triggers CORS preflight failures that look like backend errors.

4.8 macOS-specific troubleshooting

SYMPTOM	CAUSE AND FIX
<code>E: Unable to locate package mongodb-org-server</code> during a build	Building natively on Apple Silicon. Add <code>--platform linux/amd64</code> .
Container is slow on an M-series Mac	Expected — the image is <code>linux/amd64</code> and runs under emulation. It works; it is just not native.
Blank page at :8382, container healthy	<code>frontend/build</code> is empty. Build the bundle (4.4), then <code>docker compose restart lama</code> .
<code>brew services start mongodb-community</code> → "Formula not installed"	Use the versioned name: <code>mongodb-community@8.0</code> .
<code>library load disallowed by system policy</code>	macOS quarantined a downloaded file. <code>xattr -dr com.apple.quarantine .</code> then rebuild <code>.venv</code> .
<code>pull access denied for lama:local</code>	<code>.env</code> pins a local image you have not built. Use Option A in 4.6.
<code>KeyError: 'MONGO_URL'</code> at backend startup	<code>backend/.env</code> is missing or not next to <code>server.py</code> . Re-run <code>./scripts/setup.sh</code> .

Installation on Linux

Linux is the platform LAMA's container is actually built for — the image is `linux/amd64`, so there is no emulation and no platform flag. Two Linux specifics need attention: Docker's **rootless/group setup**, and the fact that **Ollama binds to loopback by default**, which makes it invisible to the container.

Commands below use `apt` (Debian/Ubuntu). Translate package names for Fedora/RHEL (`dnf`) or Arch (`pacman`) as needed.

5.1 Install the prerequisites

Path A (Docker)

Install Docker Engine plus the Compose plugin from Docker's own repository — distribution packages are frequently too old for `docker compose` (v2):

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker "$USER"
newgrp docker          # or log out and back in
```

Verify — **without** `sudo`, which confirms the group took effect:

```
docker version --format '{{.Server.Version}}'
docker compose version
```

Path B (local terminal)

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git curl netcat-openbsd
```

`netcat-openbsd` is not optional decoration: `doctor.sh` and `run-backend.sh` use `nc -z` to check whether MongoDB is listening.

Node 20+ — distribution packages are usually too old:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo corepack enable          # provides yarn 1.22.22
```

MongoDB 7/8 — from MongoDB's own repository:

```
curl -fsSL https://www.mongodb.org/static/pgp/server-8.0.asc \
| sudo gpg --dearmor -o /usr/share/keyrings/mongodb-server-8.0.gpg
echo "deb [signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg] https://repo.mongodb.org/apt/ubuntu
$(lsb_release -cs)/mongodb-org/8.0 multiverse" \
| sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list
sudo apt update && sudo apt install -y mongodb-org
sudo systemctl enable --now mongod
```

Verify everything:

```
python3 --version      # 3.11 or newer
node --version         # 20 or newer
yarn --version         # 1.22.x
nc -z 127.0.0.1 27017 && echo "Mongo up"
```

Optional toolchains, only if you will generate code for those targets:

```
sudo apt install -y maven gradle golang-go
# JDK 25 (Temurin) and .NET SDK 10 come from their vendors' repositories –
# distribution packages ship older majors that cannot compile what the
# spring-boot-4 and dotnet-10 targets emit.
```

5.2 Get the code and create the paths Compose expects

```
git clone <your-repo-url> lama
cd lama

mkdir -p certs hf_cache wheels frontend/build
touch certs/corp-ca.pem
```

A bind-mount whose source is missing is created by Docker as a **directory**, even when a *file* was expected — so a missing `corp-ca.pem` silently becomes a folder. Create them before the first `up`.

PATH	WHAT IT IS	IF YOU DO NOT USE THE FEATURE
<code>frontend/build</code>	The compiled UI bundle Nginx serves	Must be built — see 5.3
<code>certs/corp-ca.pem</code>	Your corporate root CA	An empty file is fine
<code>hf_cache</code>	Offline models for confidence scoring	An empty folder is fine
<code>wheels</code>	Offline LangGraph wheels	An empty folder is fine

5.3 Build the UI bundle

`frontend/build/` is gitignored, and Compose mounts your local copy **over** the one baked into the image. If it is empty you get a **blank page at :8382 with a healthy container**.

Path A (no Node installed):


```
docker run --rm \
-v "$(pwd)/frontend:/app" -w /app \
-e CI=true -e DISABLE_ESLINT_PLUGIN=true \
node:24-bookworm-slim \
sh -c 'corepack enable && yarn install --frozen-lockfile && yarn build'
```

NOTE

If your Docker is **rootless**, the files land owned by your user already. On a rootful daemon the container writes as `root`, so afterwards run `sudo chown -R "$USER:$USER" frontend/node_modules frontend/build` if you also intend to work on the frontend natively.

Path B (Node installed):

```
cd frontend && yarn install && yarn build && cd ..
```

5.4 Configure `.env` (Path A)

```
OPENROUTER_API_KEY=sk-or-...
LAMA_JWT_SECRET=<your own random 32+ character string>
LAMA_FACTORY_MODE=
LAMA_DEFAULT_MODEL=
```

SETTING	WHY
<code>LAMA_JWT_SECRET</code>	Signs login tokens. Setting it here is not enough for the container — see 6.4 Securing the login .
<code>LAMA_FACTORY_MODE=</code>	Blank. The committed default is <code>cli</code> , which requires the optional <code>droid</code> binary inside the container; without it, every LLM call fails.
<code>LAMA_DEFAULT_MODEL=</code>	Leave empty . Any value here overrides per-agent tier routing for <i>every</i> call.

Reaching a host Ollama from the container — the Linux gotcha

Docker Desktop on macOS and Windows provides `host.docker.internal` automatically. Plain Docker Engine on Linux does not, so `docker-compose.yml` maps it explicitly to the host gateway:

```
extra_hosts:
- "host.docker.internal:host-gateway"
```

That half is already done for you. The **other** half is Ollama itself: by default it listens on `127.0.0.1` only, which the container cannot reach. Bind it to all interfaces:

```
sudo systemctl edit ollama
```

Add:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

Then:

```
sudo systemctl restart ollama
curl http://$(hostname -I | awk '{print $1}'):11434/api/tags    # should answer
```

And in `.env`:

```
LAMA_OLLAMA_BASE_URL=http://host.docker.internal:11434/v1
```

IMPORTANT

Do not expose port 11434 to the public internet. `OLLAMA_HOST=0.0.0.0` makes the daemon reachable from your whole network. If the machine is not behind a firewall, restrict it with `ufw` or bind Ollama to the Docker bridge address instead of `0.0.0.0`.

5.5 Start it (Path A)

The committed `.env` pins `LAMA_IMAGE=lama:local` with `LAMA_PULL_POLICY=never`, so Compose expects an image **you** built. Pick one:

```
# Option A – pull the published image (fastest)
LAMA_IMAGE=mishramesh/lama:latest LAMA_PULL_POLICY=always docker compose up -d

# Option B – thin local build (adds git + dulwich on the published base)
docker build -f Dockerfile.local -t lama:local . && docker compose up -d

# Option C – full build from source
docker build -t lama:local . && docker compose up -d
```

On `x86_64` Linux no `--platform` flag is needed. On **arm64 Linux** (Graviton, Ampere, Raspberry Pi) you must still build with `--platform linux/amd64` and run under emulation — a native arm64 build fails at the MongoDB layer, which publishes no arm64 Debian packages on that channel.

```
docker compose logs -f lama
curl http://127.0.0.1:8382/health    # → {"ok":true}
xdg-open http://localhost:8382
```

First boot takes roughly 40 seconds while it seeds MongoDB, the prompt library and the agent configuration.

5.6 Path B — running natively on Linux

```
./scripts/setup.sh    # creates .venv, installs deps, writes backend/.env
./scripts/doctor.sh   # read-only check of Mongo, Ollama, toolchains
```

`setup.sh` prefers `backend/requirements-dev-macos.txt` — despite the name it is simply the **portable** dependency set (no GPU/ML stack) and works on Linux. If you want the exact container dependency set instead, including the HuggingFace stack used by the offline confidence engine:

```
./venv/bin/python -m pip install -r backend/requirements.txt
```

`setup.sh` writes `backend/.env` with a generated JWT secret. Review it and add your provider key. Then, in two terminals:

```
./scripts/run-backend.sh      # API → http://127.0.0.1:8000/api/health
```

```
./scripts/run-frontend.sh     # UI  → http://localhost:3000
```

One line in the scripts is macOS-flavoured: when MongoDB is unreachable they suggest `brew services start mongodb-community@8.0`. On Linux the equivalent is `sudo systemctl start mongod`. The check itself is correct on both.

5.7 Running LAMA as a server for a team

If this Linux box is a shared server rather than a workstation:

```
# start on boot, survive reboots — already set by the compose file
docker compose up -d          # restart: unless-stopped

# confirm it comes back
sudo systemctl enable docker
```

Then tighten these, which are permissive by default for local use:

SETTING	DEFAULT	CHANGE TO
LAMA_JWT_SECRET	a placeholder in the committed <code>.env</code>	a real random secret
Superadmin password	<code>lama-admin-2026</code>	set <code>LAMA_SUPERADMIN_PASS</code> before first boot
CORS_ORIGINS	<code>*</code>	your actual UI origin
Port <code>8382</code>	bound to all interfaces	put a TLS-terminating reverse proxy in front, or bind to <code>127.0.0.1:8382</code> in the <code>ports:</code> mapping

Both the JWT secret and the superadmin password need a small edit to `docker-compose.yml` to reach the container at all — see [6.4 Securing the login](#).

5.8 Linux-specific troubleshooting

SYMPTOM	CAUSE AND FIX
<code>permission denied while trying to connect to the Docker daemon socket</code>	Your user is not in the <code>docker</code> group. <code>sudo usermod -aG docker "\$USER"</code> then <code>newgrp docker</code> .
<code>docker compose</code> → "is not a docker command"	You have Compose v1. Install the Compose plugin, or use Docker's own repository (5.1).
Container cannot reach Ollama on the host	Ollama is bound to loopback. See 5.4 — set <code>OLLAMA_HOST=0.0.0.0</code> .
<code>nc: command not found</code> from <code>doctor.sh</code>	<code>sudo apt install netcat-openbsd</code> .
<code>frontend/build</code> owned by root after the container build	Rootful Docker daemon. <code>sudo chown -R "\$USER:\$USER" frontend/build frontend/node_modules</code> .
<code>mongod</code> won't start, <code>Permission denied</code> on <code>/var/lib/mongodb</code>	SELinux or a stale data directory. <code>sudo chown -R mongodb:mongodb /var/lib/mongodb</code> and check <code>journalctl -u mongod</code> .
Blank page at :8382, container healthy	<code>frontend/build</code> is empty. Build the bundle (5.3), then <code>docker compose restart lama</code> .
Port 8382 already bound	Change the host side only: <code>ports: - "9382:8382"</code> .

First boot and sign-in

6.1 Confirm it is alive

PATH	HEALTH CHECK	APPLICATION
Docker	<code>http://localhost:8382/health</code> → <code>{"ok":true}</code>	<code>http://localhost:8382</code>
Local terminal	<code>http://127.0.0.1:8000/api/health</code> → <code>{"ok":true}</code>	<code>http://localhost:3000</code>

On the Docker path, first boot takes about 40 seconds. `docker compose ps` shows the container's health; until it reports `healthy`, a browser may see a connection refused, which is expected.

Two more endpoints are worth bookmarking:

URL	WHAT IT TELLS YOU
<code>.../api/health/providers</code>	Which LLM provider is actually live — the first thing to check when generation fails
<code>.../docs</code> <i>(local path only)</i>	The interactive API reference

6.2 Sign in

LAMA is multi-tenant and every request is authenticated. The first boot seeds a super-admin account:

	DEFAULT	OVERRIDE WITH
Username	<code>superadmin</code>	<code>LAMA_SUPERADMIN_USER</code>
Password	<code>lama-admin-2026</code>	<code>LAMA_SUPERADMIN_PASS</code>

This is a published default. Change it before anyone else can reach the install — see 6.4.

A super-admin can see every tenant and reach the Admin pages (tenant, user and dashboard management). Ordinary users are scoped to one tenant.

6.3 What the first boot created

The seed is idempotent — it runs on every backend start and never overwrites what is already there. On an empty database it creates:

- The **PMIS Migration Pilot** reference project (PHP 8 / CodeIgniter 4 / MariaDB → FastAPI / Python 3.12 / PostgreSQL). Use it to learn the UI without importing your own code.
- The prompt library.
- The agent configuration and its routing tiers.

6.4 Securing the login

Two settings are documented as `.env` values but **do not reach the container through `.env` alone**. Compose only forwards variables it names explicitly in its `environment:` block, and neither `LAMA_JWT_SECRET` nor the `LAMA_SUPERADMIN_*` family is listed there. Setting them in the repository-root `.env` has no effect on a Docker install: the backend falls back to a shared development JWT secret and the published default password.

On the Docker path, add them to the `environment:` block of `docker-compose.yml`, alongside the keys already there:

```
environment:
  # ... existing keys ...
  LAMA_JWT_SECRET:      ${LAMA_JWT_SECRET:-}
  LAMA_SUPERADMIN_USER: ${LAMA_SUPERADMIN_USER:-}
  LAMA_SUPERADMIN_PASS: ${LAMA_SUPERADMIN_PASS:-}
  LAMA_SUPERADMIN_RESET: ${LAMA_SUPERADMIN_RESET:-}
```

Then put the values in `.env` and recreate the container:

```
# .env
LAMA_JWT_SECRET=<a random string, 32+ characters>
LAMA_SUPERADMIN_USER=youradmin
LAMA_SUPERADMIN_PASS=<a real password>
```

```
docker compose up -d      # recreates with the new environment
```

On the local-terminal path this is simpler: `backend/.env` is loaded straight into the process environment, so putting the same four keys there is enough. `./scripts/setup.sh` already generates a strong `LAMA_JWT_SECRET` for you.

Resetting a forgotten superadmin password

The seed will not overwrite an existing account, so changing `LAMA_SUPERADMIN_PASS` alone does nothing once the user exists. Opt in to the reset explicitly:

```
LAMA_SUPERADMIN_RESET=1 LAMA_SUPERADMIN_PASS=<new password> docker compose up -d
```

The reset applies on the next backend start and logs `Super-admin password reset via LAMA_SUPERADMIN_RESET=1`.

Remove `LAMA_SUPERADMIN_RESET` afterwards, or every subsequent restart re-applies that password.

6.5 Configure an LLM provider — do this before anything else

Nothing in LAMA generates without a provider. There are no hard-coded vendor defaults: an unconfigured install raises an error rather than guessing.

1. Open **Console** from the sidebar.
2. On the **Providers** tab, click **Add Provider**. Presets exist for OpenRouter, Anthropic, OpenAI, Azure, Gemini, Groq and Ollama, plus a generic OpenAI-compatible option.
3. Paste your API key and click **Fetch Models** to populate the catalogue.
4. Click **Test**. A green result means the key and base URL are correct.

5. Set the **routing tiers**. This is the step people skip, and generation fails without it.

Routing tiers

Every agent in LAMA declares a complexity tier. The Console maps each tier to an actual model, so one provider row serves the whole product.

TIER	TYPICAL USE	PICK A MODEL THAT IS
trivial	tiny classifications, routing decisions	cheapest available
low	chat, sequence diagrams, small edits	fast and cheap
medium	SRS generation, service code, most work	balanced
high	architecture recommendation, SRS regeneration, gap recovery	strong
critical	the agents that write and judge code — Transformer coder/verifier/planner, CodeGen backend/frontend coders, the regenerator	your best coding model
reasoning	diagnosis when something is stuck	a reasoning model — this is a <i>sideways</i> step from <code>critical</code> , not a rung above it

A provider row written before the six-tier model existed (with only `low` / `medium` / `high`) still routes: missing tiers fall back down a defined chain. But filling in `critical` explicitly is worth it, because that is the tier doing your actual code generation.

6. Confirm on **Console → Providers** that the provider shows as active, then reload `.../api/health/providers` and check it reports a healthy generation provider.

WATCH OUT

A saved provider with a bad key does not announce itself. It stays `is_active` and every call returns 401, which surfaces downstream as empty sections rather than an auth error. If generation produces nothing, click **Test** before debugging anything else.

6.6 Optional: enable semantic search

Without a vector store, LAMA falls back to plain-text context — everything works, retrieval is just less precise, and it degrades **silently**.

Embedded, no server (local path, simplest): set in `backend/.env`

```
QDRANT_PATH=./qdrant_local
```

A real Qdrant server: set `QDRANT_URL` (and `QDRANT_API_KEY`). `QDRANT_URL` wins over `QDRANT_PATH`.

Set **one or the other**. With neither, the whole subsystem is inert.

6.7 Behind a corporate TLS-intercepting proxy

If your organisation runs Zscaler, Netskope or similar, unmodified TLS verification will fail and the error usually surfaces as something misleading — `CERTIFICATE_VERIFY_FAILED`, or even "no provider configured".

Prefer a real CA bundle over disabling verification:

1. Drop your corporate root certificate at `certs/corp-ca.pem`. Compose already bind-mounts it and points Node at it via `NODE_EXTRA_CA_CERTS`.
2. For the Python side, set `LAMA_CA_BUNDLE` to the same certificate.
3. Only as a development last resort: `LAMA_DISABLE_SSL_VERIFY=true`. Never in production — it is a trust decision, not a convenience flag.

Using LAMA

7.1 The workspace

AREA	WHAT IS THERE
Left sidebar	The project switcher at the top, then the stages of the active project, then the admin and tool pages. Collapse it with the chevron; the state is remembered.
Main panel	The active stage. Most stage pages are split into resizable panels — drag the thin divider to give more room to chat, artifacts, diagrams or code.
MiniConsole	A live tail of backend logs, shown on long-running pages. Follow/Pause controls the scroll; Copy puts the buffer on your clipboard, which is the single most useful thing to attach to a support request.
Floating chat	A chat overlay, available on the stage pages, grounded in the project's knowledge base.

Stage badges

Each stage in the sidebar carries a badge: **locked**, **ready/available**, **frozen**, or **skipped**. Clicking a locked stage shows a message and does not open it — that is the pipeline gate doing its job, not a bug.

Skipping a stage

Data Model, Architecture and Living have a **Skip** option in their overflow menu. Skipping records a skip context and unlocks the next stage. Discovery and Code Generation cannot be skipped.

7.2 Creating a project

Click **New Project** in the sidebar. You pick a **type** first, then a name. The type decides which stages appear.

TYPE	WHAT IT RUNS	LANDING PAGE
Legacy Modernization	The full 5-stage pipeline	Discovery
Gap Analyzer	Input Files → Knowledge Base → Gap Report	Gap Analyzer
Technology Transformer	Input Files → Knowledge Base → Transformed	Transformer
Direct Transform	Configure → Transform → Output	Direct Transform

[Part 1.4](#) describes what each type is for and what it can do; this section covers the mechanics of creating one.

For the first three types the form asks for a name only — the target stack is chosen later, once LAMA has seen the code. Direct Transform asks for its folders and stacks on its own first stage.

WATCH OUT

Direct Transform has no Knowledge Base stage. It is folder-path driven and builds no KB, so that stage would never become active. Its three stages are **Configure → Transform → Output**.

Learning on the reference pilot

The seeded **PMIS Migration Pilot** project is there so you can walk the whole pipeline before importing anything of your own. Switch to it, configure a provider, and follow 7.3 onward.

7.3 Stage 1 — Discovery

Discovery turns legacy evidence into a knowledge base and a frozen SRS. This is where architects and SMEs spend most of their review time, and the quality of everything downstream is decided here.

Step 1 — Import the legacy evidence

Four ways in, all on the Discovery page:

METHOD	USE WHEN	NOTE
Upload Files	You have a folder or ZIP to hand	ZIPs are parsed in memory. Uploads are additive .
Scan Local Folder	The code is on the machine running LAMA	Takes an absolute path. Replaces the existing KB by default. In Docker, the path must be one the container can see.
Clone Git Repository	The code is in Git	HTTPS URL, branch, optional username/token. Ingest continues in the background; the UI polls.
Live Data Sources	You can reach the running system	Connect a database, or register an application URL, to add schema and endpoint evidence.

Some paths are skipped deliberately and will not appear in your file count:

```
node_modules  .git  vendor  __pycache__
*.bak  *.save  *_bkp  *_old  *_backup  *.php_*
```

If a file you expected is missing, check that pattern list before assuming an ingest failure.

Target Stack. Also on Discovery: review the suggested target stacks and apply one. That choice is saved to the project and shapes every downstream prompt.

Step 2 — Build the Knowledge Base

Click **Build Knowledge Base**. A progress dialog opens and reports each phase.

PHASE	PRODUCES
Parsers	File records and text chunks
Tech detection	The stack fingerprint (language, framework, database)

PHASE	PRODUCES
OWL extraction	Entities — classes, routes, tables, columns, roles
TOON serialization	The compact context format used by chat and SRS
Business ontology / graph	Business domains, entities and relationships
Vector store	RAG-ready embedded chunks (only if Qdrant is configured)

When it finishes, the metrics row updates: **Files, Entities, Chunks**. All three should be non-zero. If it fails, read the phase name in the dialog before closing it, and copy the MiniConsole buffer.

Step 3 — Interrogate the KB with chat

Open the floating chat and ask questions grounded in what was ingested — *"What are the main workflows?"*, *"Which roles can approve a claim?"*

CONTROL	WHAT IT DOES
Model dropdown	Picks the model for Discovery chat and SRS generation . Leave it on Auto to use Console routing. The choice is remembered across reloads.
SRS Edit toggle	Turns chat replies into proposed section edits you can apply to the SRS.
New session / End session	Starts or archives a rolling-memory session that survives a page refresh.

NOTE

The model you pick here drives SRS generation too. If SRS output looks unexpectedly weak, check what is selected in the chat dropdown.

Step 4 — Generate the SRS

Click **Generate SRS**. LAMA streams twelve IEEE-830 sections, sometimes several in parallel.

#	SECTION
1	Introduction
2	Overall Description
3	Actors and Use Case Inventory
4	Specific Requirements
5	Detailed Use Cases
6	External Interfaces
7	Non-Functional Requirements
8	Integration Requirements

#	SECTION
9	Validation and Verification
10	Traceability Matrix
11	Appendices
12	Entity Relationship Model

- **Pause / Resume / Cancel** control a long run. Pause takes effect at the next section checkpoint.
- A network interruption is recoverable — the UI polls the backend job and refreshes when it completes.
- **Regenerate** one section when only that section is weak, rather than rerunning all twelve.
- **Edit** a section to correct it by hand. Saving increments the SRS version.
- **Export PDF** from the SRS action menu.

WATCH OUT

Treat any section containing ⚠ EVIDENCE GAP or NOT_EVIDENCED as a review item. It means the model declined to invent something the source did not support — which is the behaviour you want, and a signal that the KB is missing evidence.

Step 5 — Check confidence

The **Confidence** pill sits near the freeze controls. Click it to open a per-section breakdown; for Discovery the rows line up with the twelve SRS sections.

COLUMN	HOW TO READ IT
Score	≥95 is freeze-friendly. Lower rows need inspection.
Route	How the score was reached — see below.
Rationale / gaps	A short explanation of what is missing.

ROUTE	MEANING	WHAT TO DO
hf_accept	Coverage is high and no contradictions found; typically scores 96	Usually safe. Still read business-critical sections yourself.
hf_reject	Coverage is low or middling	Open the section, add evidence-backed content or regenerate, then Recompute.
hf_reject_hardstop	Contradictions exceeded the hard-stop threshold	Do not freeze. Resolve the contradiction in the SRS or the KB first.
engine_unavailable	The scoring engine could not load	An installation problem, not a content problem — see 9.3.
Every row at 55%	The offline encoders are unavailable	See 9.3.
Every row at 0%	The scoring engine is missing entirely	See 9.3.

Recompute after: the first SRS generation, any manual edit, any section regeneration, any KB rebuild — and always before freezing.

Confidence scoring is deliberately **token-free** in the default configuration: it uses two small local encoders, not your LLM budget.

Step 6 — Freeze

When every section is populated and the low-confidence rows have been reviewed, click **Freeze** and type **FREEZE** exactly. That writes the Discovery Stage Context and unlocks Data Model.

The typed confirmation is intentional friction. Freezing is what downstream stages build on.

To go back, use Unfreeze / Reset and type **RESET** where asked.

7.4 Stage 2 — Data Model

Turns the frozen SRS and KB into the target database design. If Discovery is not frozen, the page shows a locked banner and generation refuses.

ACTION	PRODUCES
Generate Entity Graph	The ER diagram at the top of the page
Generate OLTP	Normalised PostgreSQL DDL for the operational schema
Generate OLAP	A star schema for BI and natural-language-to-SQL
Generate Bus Matrix	The fact × dimension matrix
Generate All Scripts	Three migration scripts: legacy→OLTP, OLTP→OLAP, and a migration test

Each artifact supports **Edit**, **Save**, **Freeze** and **Download** individually. Freezing OLTP and OLAP is what drives the cascade into Architecture.

The Data Model chat understands change requests: ask for a schema change and, when it detects a DDL change, it offers an **Apply** button that writes it into the OLTP or OLAP artifact.

7.5 Stage 3 — Architecture

Consumes the Data Model context and produces the service design.

TAB	ACTION	PRODUCES
Service Map	Recommend	A proposed service decomposition
Service Map	Approve / Merge / Unmerge	The approved service set that CodeGen will build
HLD	Generate	High-level design, with rendered Mermaid diagrams
LLD	Generate	Low-level design per service
Sequence Diagrams	Generate	Mermaid sequence diagrams per use case

TAB	ACTION	PRODUCES
API Contracts	Generate	OpenAPI 3.1 YAML per service

WATCH OUT

Architecture does not stream. It runs background jobs and polls every 2 seconds, because production ingress commonly times out at 60 seconds and a long-lived stream would be cut. A progress bar that updates every couple of seconds is the expected behaviour, not a stall.

Get the **Service Map right before generating anything else** — HLD, LLD, contracts and all of CodeGen are built from the approved service set.

The Architecture chat recognises change requests for the HLD, the architecture as a whole, and service addition or removal, and offers to apply them.

7.6 Stage 4 — Code Generation

Turns the frozen architecture into target source trees. This is the deliverable stage; review before you push.

CONTROL	WHAT IT DOES
Generate All	Generates every approved service
Generate selected service	Regenerates one service only
Gap Recovery (Backend / Frontend)	Finds and repairs legacy-parity gaps in already-generated files
Auto-Validate & Improve	Scores generated files across six axes, repairs the worst, and loops until a threshold is met
File tree → Monaco editor	Read and hand-edit any generated file; Save persists it
Download ZIP	The whole generated tree as a ZIP
Export to Disk	Writes the tree to a host folder — by default a sibling of the LAMA repository
Push to GitHub	Pushes using the credentials saved in GitHub Settings
Freeze	Writes the CodeGen context and unlocks Living

Notes worth knowing:

- An **empty ZIP** almost always means the Architecture service map was never approved and frozen. Go back to Stage 3.
- **Export to Disk** writes to whatever host directory is mapped to `/lama-export`. By default that is the parent of the LAMA repository; override with `LAMA_EXPORT_HOST_DIR` in `.env`.
- The file tree is **deliberately flattened** rather than deeply recursive. That is a known workaround, not a display bug.
- GitHub push needs a personal access token configured under **GitHub Settings** first.

7.7 Stage 5 — Living System

The "keep the SRS honest" stage. Less generative, more operational.

FEATURE	WHAT IT DOES
Selenium Tests	Generates JUnit 5 + Selenium acceptance tests from the use cases
JMeter Plans	Generates performance test plans per persona
Drift Detector	Paste live signals from the running system; get a report on where it has drifted from the frozen SRS
SRS Diff	Compare two SRS snapshots; get added/removed/modified requirements plus a recommendation of what to regenerate downstream
Accuracy Report	Scores the KB, SRS and artifacts, with deep links back to whatever needs regenerating

Treat Living as an evolving capability: it generates and manages these artifacts well, but it is not a full APM product.

7.8 The three non-pipeline project types

None of these requires a frozen upstream stage, and none of them uses the five-stage sidebar. [Part 1.4](#) covers what each is for; this section is the operating detail.

Gap Analyzer

Create a project of type "Gap Analyzer", or open it from the sidebar.

1. **Input Files** — upload the source code plus the SRS/FRS document to check it against.
2. **Knowledge Base** — LAMA extracts UI → API → DB traceability and a requirement list.
3. **Gap Report** — a coverage matrix and a severity-ranked list of requirements the code does not implement.

Has its own freeze/unfreeze and its own export, independent of the pipeline.

Technology Transformer

Create a project of type "Technology Transformer", or open it from the sidebar.

A code-transform super-agent. Point it at a source tree, pick a target stack, and it generates the migrated code, then compiles it and repairs what fails.

The repair loop escalates through four rungs — coder, then a DevOps expert, then the DevOps expert with the raw build log, then a full regenerator — capped at five iterations.

NOTE

The build gate is a report, not a permission. `compile_green`, `production_ready` and the DevOps audit panel tell you the state of the build. They do **not** block download or GitHub push. You can always collect your own code, whether or not the loop reached a green build.

Direct Transform

Create a project of type "Direct Transform".

Its three stages are **Configure → Transform → Output** — there is no Knowledge Base stage, because it builds none.

1. **Configure** — pick the source folder and destination with the server-side picker, then pick a **source stack** and a **target stack** independently.
2. **Transform** — run the job and watch the engine and agent event stream. You can pause, resume, or roll back.
3. **Output** — the reports and the per-file transform record.

FAMILY	AVAILABLE STACKS
Backend	Helidon MicroProfile, Helidon SE, Spring Boot 4.1 (Java 25 LTS), Spring Boot 3.x (Java 21), .NET 10 LTS, JSP / Jakarta Pages 4.0, Apache Struts, EJB 3.x
Frontend	React 19, Angular 22, jQuery
Database	Oracle (SQL/PL-SQL), PostgreSQL 18, PostgreSQL 15

Some pairs have a **deterministic** transformer (a hand-written plugin — for example Helidon MP → Spring Boot 3, or Oracle → PostgreSQL); the rest go through a generic AI pass. The UI tells you which one your chosen pair will get before you start.

WATCH OUT

The folder picker browses the *container's filesystem*, not yours. On a Docker install it opens on the image's own `$HOME` by default and will never show your code. Point it at something bind-mounted — Compose already maps the parent of the LAMA repository to `/lama-export` : `` `ini

.env

LAMA_DCTE_WORKSPACE=/lama-export `` `

7.9 The admin pages

PAGE	WHAT IT IS FOR
Console	Providers, routing tiers, per-agent configuration and budgets, prompt preview/test, and token-spend reporting. See 6.5.
Prompt Library	Every seeded prompt, editable. Direct Transform's prompts are the <code>dcte.*</code> entries — note that only the fixed half is editable; the stack-specific half is computed per source/target pair.
Ontology Studio	The business-domain graph, available once the KB is built.
GitHub Settings	The credentials CodeGen and Transformer use to push.
Audit Log	Every state-changing action, with who and when.
Integrations	The integration catalogue and templates.
Admin Dashboard (super-admin only)	Tenants, users, and the cross-tenant view.

Console → Usage

Worth checking regularly. It groups token spend by provider, model, stage and agent over a date window, which is how you find the agent that is quietly costing you the most.

Day-to-day operations

8.1 Lifecycle — Docker

```
docker compose up -d      # start, or apply .env / compose changes
docker compose stop       # stop, keep containers and data
docker compose start      # start again
docker compose restart lama # restart in place
docker compose ps         # what is running, and its health
docker compose down       # remove the container, KEEP the volumes
docker compose down -v    # remove the container AND DELETE ALL DATA
```

IMPORTANT

`docker compose down -v` destroys every project, knowledge base, SRS and generated file. Back up first — see 8.5.

8.2 Lifecycle — local terminal

Both processes run in the foreground; `Ctrl-C` stops them.

```
./scripts/run-backend.sh    # API on :8000
./scripts/run-frontend.sh   # UI on :3000
./scripts/doctor.sh         # read-only environment check
./scripts/clear-caches.sh   # clear build and dependency caches
```

MongoDB is managed separately:

PLATFORM	START	STOP
macOS	<code>brew services start mongodb-community@8.0</code>	<code>brew services stop mongodb-community@8.0</code>
Linux	<code>sudo systemctl start mongod</code>	<code>sudo systemctl stop mongod</code>
Windows	<code>Start-Service MongoDB</code>	<code>Stop-Service MongoDB</code>

8.3 Logs

```
docker compose logs -f lama      # everything, following
docker compose logs --tail=200 lama # last 200 lines
docker compose logs --since=10m lama # last 10 minutes
```

Per-process, inside the container:

```
docker compose exec lama tail -f /var/log/backend.out.log
docker compose exec lama tail -f /var/log/backend.err.log
docker compose exec lama tail -f /var/log/mongodb.err.log
docker compose exec lama tail -f /var/log/nginx.err.log
```

In the UI, the **MiniConsole** shows the same backend stream with a Copy button — the fastest way to capture context for a support request.

8.4 Applying changes

`./backend` and `./frontend/build` are bind-mounted, so most edits need no image rebuild.

```
# after a Python change under backend/
docker compose restart lama

# after a React change – rebuild the bundle, then restart
docker run --rm -v "$(pwd)/frontend:/app" -w /app -e CI=true -e DISABLE_ESLINT_PLUGIN=true \
  node:24-bookworm-slim sh -c 'corepack enable && yarn build'
docker compose restart lama
```

Only a change to `requirements.txt`, the `Dockerfile`, or the bundled toolchains needs a real image rebuild.

8.5 Backup and restore

All of your work lives in one MongoDB volume.

```
# back up
docker compose exec lama mongodump --db lama --archive=/tmp/lama.archive
docker compose cp lama:/tmp/lama.archive ./lama-backup.archive

# restore
docker compose cp ./lama-backup.archive lama:/tmp/lama.archive
docker compose exec lama mongorestore --archive=/tmp/lama.archive --drop
```

Do this before any image upgrade.

What lives where

VOLUME	HOLDS	SURVIVES DOWN
<code>lama_mongo_data</code>	Every project, KB, SRS, artifact, audit log	Yes
<code>lama_hf_cache</code>	Offline models for confidence scoring	Yes
<code>lama_wheels</code>	Offline LangGraph wheels, seeded on first boot	Yes

```
docker volume ls | grep lama
docker volume rm lama_hf_cache      # drop just the model cache; re-seeds next boot
```

8.6 Upgrading

```
# 1. back up (8.5)
# 2. pull the new image
LAMA_IMAGE=mishramesh/lama:latest LAMA_PULL_POLICY=always docker compose pull
# 3. rebuild the UI bundle (8.4)
# 4. recreate
docker compose up -d
```

Named volumes survive image upgrades, so your projects come back untouched.

8.7 Inside the container

```
docker compose exec lama bash                # a shell
docker compose exec lama supervisorctl status # process states
docker compose exec lama supervisorctl restart backend
docker compose exec lama mongosh lama         # the Mongo shell

# which toolchains are present
docker compose exec lama sh -c 'javac -version; dotnet --version; node -v; mvn -v | head -1'
```

Troubleshooting

Platform-specific problems are in each OS section — [Windows 3.9](#), [macOS 4.8](#), [Linux 5.8](#). What follows applies everywhere.

9.1 It will not start

SYMPTOM	CAUSE AND FIX
Blank page, container reports healthy	<code>frontend/build</code> is empty. Build the bundle, then <code>docker compose restart lama</code> .
<code>pull access denied for lama</code>	<code>.env</code> pins <code>LAMA_IMAGE=lama:local</code> and you have not built it. Override with <code>LAMA_IMAGE=mishramesh/lama:latest LAMA_PULL_POLICY=always</code> .
Port 8382 already in use	Change the host side only in <code>docker-compose.yml</code> : <code>ports: - "9382:8382"</code> .
<code>KeyError: 'MONGO_URL'</code> (local path)	<code>backend/.env</code> is missing, or is not sitting next to <code>server.py</code> .
Backend starts then hangs (local path)	MongoDB is unreachable; the startup seed is waiting for it. Start MongoDB.
Something is already running on port 3000	A previous dev server survived. <code>lsof -nP -iTCP:3000 -sTCP:LISTEN</code> , then kill the PID.

9.2 Nothing generates

Work down this list in order — it is ordered by how often each one is the cause.

1. **Is a provider configured?** Open `.../api/health/providers`. If it reports no healthy provider, go to Console → Providers.
2. **Does the key actually work?** Click **Test** on the provider. A saved provider with a bad key stays marked active and returns 401 on every call, which surfaces as empty output rather than an auth error.
3. **Are the routing tiers filled in?** A provider with no tier mapping cannot answer a request for a tier.
4. **Is `LAMA_DEFAULT_MODEL` empty?** Any value there overrides per-agent routing for *every* call.
5. **Is `LAMA_FACTORY_MODE` blank?** The committed default is `cli`, which needs the optional `droid` binary inside the container. Without it, every call fails.
6. **Empty content from a local model?** Thinking models (the Qwen3 family) spend 300–1600 tokens reasoning before emitting anything. Give them a budget of 2000+, or use a non-thinking model such as `qwen2.5-coder:7b` for short calls.
7. **`CERTIFICATE_VERIFY_FAILED` ?** You are behind a TLS-intercepting proxy. See 6.7.

9.3 Confidence scores look wrong

PATTERN	MEANING	FIX
Every row at 55%	The offline encoders could not load	Check that <code>hf_cache</code> is populated and mounted; refresh the <code>lama_hf_cache</code> volume
Every row at 0% , route <code>engine_unavailable</code>	The LangGraph scoring engine is missing	Refresh the <code>lama_wheels</code> volume and restart the container
Scores vary run to run	Expected	Coverage depends on section wording, KB completeness and contradictions. Recompute after content changes.
A single low row	Working as designed	Open that section, add evidence-backed detail or regenerate it, then Recompute

To force a re-seed of either cache:

```
docker volume rm lama_hf_cache      # or lama_wheels
docker compose up -d
```

9.4 Stage and pipeline problems

SYMPTOM	CAUSE
A stage will not open	The stage before it is not frozen. That is the gate working. Freeze upstream, or skip it if the option is offered.
HTTP 400 from a generation call	Same cause — the upstream stage context does not exist.
Downloaded ZIP is empty	The Architecture service map was never approved and frozen.
Direct Transform's folder picker shows nothing familiar	It browses the container filesystem. Set <code>LAMA_DCTE_WORKSPACE</code> to a bind-mounted path.
Semantic search returns nothing	No vector store configured. Set <code>QDRANT_PATH</code> or <code>QDRANT_URL</code> . It degrades silently by design.
<code>toolchain missing on PATH: mvn</code>	Install the toolchain. A missing binary fails the job deliberately, so a build cannot report success without compiling.
A generated Java build fails with syntax errors on modern constructs	Your JDK is older than the target requires. The <code>spring-boot-4</code> target emits Java 25.

9.5 Authentication

SYMPTOM	CAUSE
<code>401 Unauthorized</code>	Expected when not signed in — LAMA is multi-tenant with JWT auth.

SYMPTOM	CAUSE
Logged out on every restart	<code>LAMA_JWT_SECRET</code> is not reaching the backend, so it regenerates a fallback. See 6.4.
Forgot the superadmin password	Reset it explicitly — see 6.4.

9.6 Getting help

Collect these before asking:

```
docker compose ps           # health
docker compose logs --tail=200 lama  # recent logs
curl http://127.0.0.1:8382/health
```

Plus the **MiniConsole** → **Copy** buffer from the page where it failed, and the stage and artifact you were generating.

Reference

10.1 Settings you are most likely to change

These go in the repository-root `.env` for the Docker path, or `backend/.env` for the local path.

VARIABLE	WHAT IT DOES
<code>OPENROUTER_API_KEY</code>	Your LLM key. Not needed if you configure providers in the Console instead.
<code>LAMA_JWT_SECRET</code>	Signs login tokens. Needs a compose edit on Docker — see 6.4.
<code>LAMA_DEFAULT_MODEL</code>	Leave empty. A value here overrides per-agent tier routing for every call.
<code>LAMA_FACTORY_MODE</code>	Leave blank unless you use the Factory <code>droid</code> CLI. The committed default is <code>cli</code> .
<code>QDRANT_PATH</code>	Embedded on-disk vector store — no server needed
<code>QDRANT_URL</code> / <code>QDRANT_API_KEY</code>	A real Qdrant server. <code>QDRANT_URL</code> wins over <code>QDRANT_PATH</code> .
<code>LAMA_OLLAMA_BASE_URL</code>	Your Ollama endpoint. From a container: <code>http://host.docker.internal:11434/v1</code>
<code>LAMA_DCTE_WORKSPACE</code>	Where Direct Transform's folder picker opens. Must be a path the container can see.
<code>LAMA_EXPORT_HOST_DIR</code>	Host directory that CodeGen's "Export to Disk" writes into
<code>LAMA_CONFIDENCE_ENGINE</code>	<code>langgraph</code> , <code>fabric</code> , or blank for automatic. Pin it explicitly to keep scoring deterministic.
<code>MONGO_URL</code> / <code>DB_NAME</code>	The database. Defaults are correct for both paths.
<code>CORS_ORIGINS</code>	Wide open (<code>*</code>) by default. Tighten for a shared server.
<code>LAMA_CA_BUNDLE</code>	Corporate root CA for the Python side
<code>LAMA_DISABLE_SSL_VERIFY</code>	Development last resort only. Never in production.

10.2 Ports

PORT	USED BY	PATH
<code>8382</code>	The whole application behind Nginx	Docker
<code>8000</code>	The FastAPI backend	Local terminal
<code>3000</code>	The React dev server	Local terminal
<code>27017</code>	MongoDB	Local terminal (bundled in Docker)

PORT	USED BY	PATH
11434	Ollama, if you use it	Both
6333	Qdrant, if you run a server	Both

10.3 Glossary

TERM	MEANING
Artifact	Any generated, versioned, freezable output — an SRS section, a DDL file, a service map, a generated source file
Bus Matrix	The fact × dimension grid that shows which dimensions each fact table shares
Confidence	A score, per artifact section, of how well the generated content is supported by the ingested evidence
DCTE / Direct Transform	The folder-driven, project-free transformation engine
Deterministic plugin	A hand-written transformer for a specific stack pair, as opposed to a generic AI pass
Drift	Divergence between the frozen SRS and what the running system actually does
Freeze	Locking a stage's artifacts and writing the Stage Context that the next stage builds on
KB (Knowledge Base)	Everything LAMA extracted from your legacy source: files, chunks, entities, relationships
OLAP	The analytical star schema
OLTP	The operational, normalised schema
Ontology	The business-domain model extracted from the code
Parity gap	Legacy behaviour that the generated code does not yet implement
Routing tier	The complexity band (trivial ... reasoning) that decides which model serves a given agent
SRS	Software Requirements Specification — the 12-section IEEE-830 document Discovery produces
Stage Context	The frozen handoff record between one stage and the next
Tenant	An isolated organisation within one LAMA install
TOON	The compact context format LAMA feeds to models in place of raw source

10.4 Further reading

DOCUMENT	COVERS
docs/DOCKER.md	The Docker-only path in more depth, macOS and Windows

DOCUMENT	COVERS
<code>docs/RUNNING.md</code>	Every command for both paths, and the test suites
<code>docs/ARCHITECTURE.md</code>	How LAMA is built internally
<code>docker/README.md</code>	Deploying to a server

Verification notes

Honest reporting of what was checked against this build, so you know which parts to trust without re-testing:

Verified against the repository and a running macOS install: the Docker path end to end, including the container-based frontend build; the behaviour of a missing bind-mount source; the seeded superadmin credentials and the `LAMA_SUPERADMIN_RESET` logic; the stage sequence and freeze gates; the Direct Transform stack catalogue; the routing tiers; the helper scripts' behaviour; and every port, endpoint and environment variable named above.

Found and corrected while writing this guide: `LAMA_JWT_SECRET` and the `LAMA_SUPERADMIN_*` variables appear in the repository-root `.env` but in **neither** `docker-compose.yml`'s `environment:` block nor either container entrypoint — so on the Docker path, setting them in `.env` alone has no effect. Section 6.4 gives the instruction that actually works. Other documents in this repository still describe the `.env`-only method; prefer 6.4.

Derived rather than executed on that platform: the Windows PowerShell commands and the Linux `apt` sequences. Both follow from the same Compose and script behaviour reproduced on macOS — specifically, the CRLF entrypoint failure and the unset-`HOME` mount resolution were reproduced deliberately — but the package-manager invocations themselves were not run on a Windows or Linux host. The Linux Ollama loopback note reflects Ollama's documented default bind address.